

Ecole

백준 문제 풀이

2025.12.06

Python

정수를 1로 만들기

**문제[084] 정수를 1로 만들기**

시간 제한 0.15초 | 난이도 실버 Ⅲ | 백준 온라인 저지 1463번

정수 X에 사용할 수 있는 연산은 다음 3가지다.

1. X가 3으로 나누어떨어지면 3으로 나눈다.
2. X가 2로 나누어떨어지면 2로 나눈다.
3. 1을 뺀다.

정수 N이 주어졌을 때 위와 같은 연산 3개를 적절히 사용해 1을 만들려고 한다. 연산을 사용하는 횟수의 최솟값을 출력하시오.

↓ 입력

1번째 줄에 1보다 크거나 같고, 106보다 작거나 같은 정수 N이 주어진다.

↑ 출력

1번째 줄에 연산을 하는 횟수의 최솟값을 출력한다.

예제 입력 1

10 // N

예제 출력 1

3

예제 입력 2

2

예제 출력 2

1



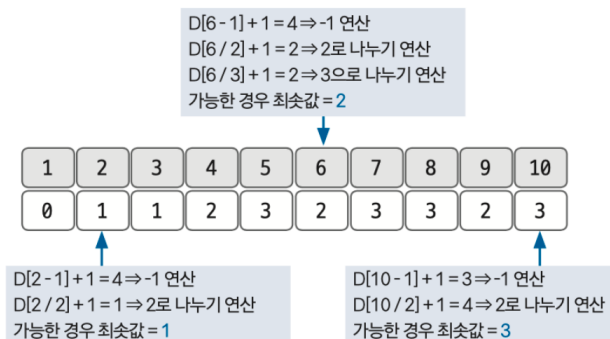
- 1 먼저 점화식의 형태와 의미를 도출한다.

$D[i]$: i 에서 1로 만드는 데 걸리는 최소 연산 횟수

- 2 점화식을 구한다.

```
D[i] = D[i - 1] + 1 // 1을 빼는 연산이 가능함
if(i % 2 == 0) D[i] = min(D[i], D[i / 2] + 1) // 2로 나누는 연산이 가능함
if(i % 3 == 0) D[i] = min(D[i], D[i / 3] + 1) // 3으로 나누는 연산이 가능함
```

- 3 점화식을 이용해 D 배열을 채운다.



- 4 $D[N]$ 을 출력한다.

$\therefore D[10] = 3$

Python

```
x=int(input()) # 수 입력받기
d=[0]*(x+1) # 1-based
for i in range(2,x+1): # 2부터 x까지 반복
    d[i]=d[i-1]+1 # 1을 빼는 연산 -> 1회 진행
    if i%2==0: # 2로 나누어 떨어질 때, 2로 나누는 연산
        d[i]=min(d[i],d[i//2]+1)
    if i%3==0: # 3으로 나누어 떨어질 때, 3으로 나누는 연산
        d[i]=min(d[i],d[i//3]+1)
print(d[x])
```


Java, JS

정수를 1로 만들기



Java

```
public class P001 { new *
    public static void main(String[] args) { new *
        Scanner sc = new Scanner(System.in);

        // 수 입력받기
        int x = sc.nextInt();

        // DP 테이블 생성 (0으로 초기화됨)
        // 1-based 인덱싱을 위해 크기를 x + 1로 설정
        int[] d = new int[x + 1];

        // 2부터 x까지 반복 (Bottom-Up)
        for (int i = 2; i <= x; i++) {
            // 1. 1을 빼는 연산 (기본 전제)
            d[i] = d[i - 1] + 1;

            // 2. 2로 나누어 떨어질 때, 2로 나누는 연산과 비교
            if (i % 2 == 0) {
                d[i] = Math.min(d[i], d[i / 2] + 1);
            }

            // 3. 3으로 나누어 떨어질 때, 3으로 나누는 연산과 비교
            if (i % 3 == 0) {
                d[i] = Math.min(d[i], d[i / 3] + 1);
            }
        }

        System.out.println(d[x]);
        sc.close();
    }
}
```

JS

```
const fs = require('fs');

// 입력 처리 (로컬 테스트 시 경로 수정 필요, 백준 제출 시 '/dev/stdin')
const input = fs.readFileSync('/dev/stdin').toString().trim();
const x = Number(input);

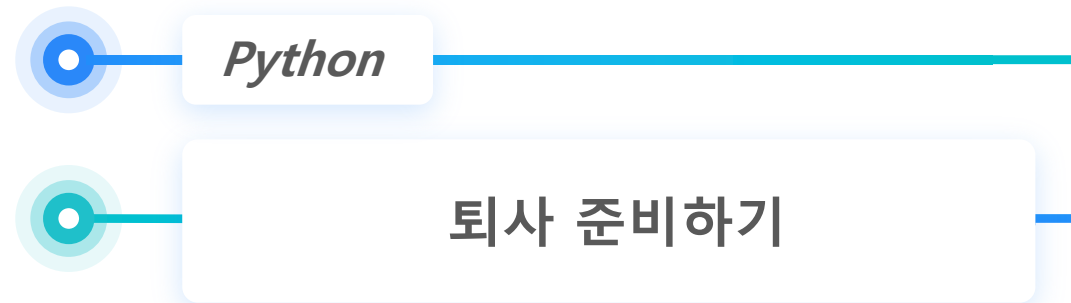
// DP 테이블 생성 (0으로 채움)
const d = new Array(x + 1).fill(0);

// 2부터 x까지 반복
for (let i = 2; i <= x; i++) {
    // 1. 1을 빼는 연산
    d[i] = d[i - 1] + 1;

    // 2. 2로 나누어 떨어질 때
    if (i % 2 === 0) {
        d[i] = Math.min(d[i], d[i / 2] + 1);
    }

    // 3. 3으로 나누어 떨어질 때
    if (i % 3 === 0) {
        d[i] = Math.min(d[i], d[i / 3] + 1);
    }
}

console.log(d[x]);
```





문제[085] 퇴사 준비하기

시간 제한 2초 | 난이도 실버 Ⅲ | 백준 온라인 저지 14501번

상담원으로 일하고 있는 백준이는 퇴사하려고 한다. 오늘부터 $N + 1$ 일째 되는 날 퇴사하기 위해 남은 N 일 동안 최대한 많은 상담을 하려고 한다. 백준이는 비서에게 최대한 많은 상담을 잡으라고 부탁했고, 비서는 하루에 1개씩 서로 다른 사람의 상담을 잡아 놓았다. 각각의 상담은 상담을 완료하는 데 걸리는 시간 T_i 와 상담했을 때 받을 수 있는 금액 P_i 로 이뤄져 있다. $N = 7$ 일 때 다음과 같은 상담 일정표를 살펴보자.

	1일	2일	3일	4일	5일	6일	7일
T_i	3	5	1	1	2	4	2
P_i	10	20	10	20	15	40	200

1일에 잡혀 있는 상담은 총 3일이 걸리며, 상담했을 때 받을 수 있는 금액은 10이다. 5일에 잡혀 있는 상담은 총 2일이 걸리며, 받을 수 있는 금액은 15이다. 상담하는 데 필요한 기간은 1일보다 클 수 있기 때문에 모든 상담을 할 수는 없다. 예를 들어 1일에 상담하면 2일, 3일에 있는 상담은 할 수 없게 된다. 2일에 있는 상담을 하면 3, 4, 5, 6일에 잡혀 있는 상담은 할 수 없다. 또한, $N + 1$ 일째에는 회사에 없기 때문에 6, 7일에 있는 상담은 할 수 없다. 퇴사 전에 할 수 있는 상담의 최대 이익은 1일, 4일, 5일에 있는 상담을 하는 것이며, 이때의 이익은 $10 + 20 + 15 = 45$ 이다. 상담을 적절히 했을 때 백준이가 얻을 수 있는 최대 수익을 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 N ($1 \leq N \leq 15$), 2번째 줄부터 N 개의 줄에 T_i 와 P_i 가 공백으로 구분돼 주어지고, 1일에서 N 일까지 순서대로 주어진다($1 \leq T_i \leq 5$, $1 \leq P_i \leq 1,000$).

↑ 출력

1번째 줄에 백준이가 얻을 수 있는 최대 이익을 출력한다.

예제 입력 1

```
7 // N
3 10
5 20
1 10
1 20
2 15
4 40
2 200
```

예제 출력 1

45

예제 입력 2

```
10
1 1
1 2
1 3
1 4
1 5
1 6
1 7
1 8
1 9
1 10
```

예제 출력 2

55

예제 입력 3

```
10
5 10
5 9
5 8
5 7
5 6
5 10
5 9
5 8
5 7
5 6
```

예제 출력 3

20

예제 입력 4

```
10
5 50
4 40
3 30
2 20
1 10
1 10
2 20
3 30
4 40
5 50
```

예제 출력 4

90



1 점화식의 형태와 의미를 도출한다.

$D[i]$: i 번째 날부터 퇴사날까지 벌 수 있는 최대 수입

2 점화식을 구한다.

$D[i] = D[i] + 1$ // 오늘 시작되는 상담을 했을 때 퇴사일까지 끝나지 않는 경우
 $D[i] = \text{MAX}(D[i + 1], D[i + T[i]] + P[i])$ // 오늘 시작되는 상담을 했을 때 퇴사일 안에 끝나는 경우

3 점화식을 이용해 D배열을 채운다.

<예제 1 상담 일정표>

	1일	2일	3일	4일	5일	6일	7일
T_i	3	5	1	1	2	4	2
P_i	10	20	10	20	15	40	200

$D[7]$

$i + T[i] = 7 + 2 = 9 > N + 1 = 8$
 해당 날에 시작되는 상담은 퇴사일까지 끝낼 수 없으므로 $D[7] = D[8] = 0$

$D[6]$

$i + T[i] = 6 + 4 = 10 > 8$
 $D[6] = D[7] = 0$

$D[5]$

$i + T[i] = 5 + 2 = 7 \leq 8$
 $D[5] = \text{MAX}(D[6], D[7] + P[5]) = 15$

$D[4]$

$i + T[i] = 4 + 1 = 5 \leq 8$
 $D[4] = \text{MAX}(D[5], D[5] + P[4]) = 35$

$D[3]$

$i + T[i] = 3 + 1 = 4 \leq 8$
 $D[3] = \text{MAX}(D[4], D[4] + P[3]) = 45$

$D[2]$

$i + T[i] = 2 + 5 = 7 \leq 8$
 $D[2] = \text{MAX}(D[3], D[7] + P[2]) = 45$

$D[1]$

$i + T[i] = 1 + 3 = 4 \leq 8$
 $D[1] = \text{MAX}(D[2], D[4] + P[1]) = 45$

4 완성된 D 배열에서 $D[1]$ 값을 출력한다.

1	2	3	4	5	6	7	8
45	45	45	35	15	0	0	0

$\therefore D[1]$ 값 출력 = 45

Python

```
N = int(input())
TP = [list(map(int, input().split())) for _ in range(N)]
dp = [0 for _ in range(N+1)]

for i in range(N-1, -1, -1):
    if i+TP[i][0]>N:
        dp[i] = dp[i+1]
    else:
        dp[i] = max(dp[i+1], dp[i+TP[i][0]]+TP[i][1])
print(dp[0])
```

Java, JS

퇴사 준비하기



Java

```

public class P002 { new *
    public static void main(String[] args) { new *
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();

        // 시간(T)과 금액(P)을 저장할 배열
        int[] T = new int[N];
        int[] P = new int[N];

        for (int i = 0; i < N; i++) {
            T[i] = sc.nextInt();
            P[i] = sc.nextInt();
        }

        // DP 테이블 생성 (N+1 크기, 0으로 자동 초기화)
        int[] dp = new int[N + 1];

        // 뒤에서부터 역순으로 반복 (N-1 부터 0까지)
        for (int i = N - 1; i >= 0; i--) {
            // 상담 기간이 남은 기간(N)을 초과하는 경우
            if (i + T[i] > N) {
                dp[i] = dp[i + 1]; // 상담 불가, 이전(사실상 다음 날짜) 값 가져옴
            } else {
                // 상담을 안 하는 경우 vs 상담을 하는 경우 중 최댓값
                // dp[i + T[i]]는 해당 상담이 끝난 시점의 누적 최대 이익
                dp[i] = Math.max(dp[i + 1], P[i] + dp[i + T[i]]);
            }
        }

        System.out.println(dp[0]);
        sc.close();
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const N = Number(input[0]);
const TP = [];

// 입력 데이터 파싱 (TP 배열 생성)
for (let i = 1; i <= N; i++) {
    TP.push(input[i].trim().split(' ').map(Number));
}

// DP 테이블 생성 (0으로 채움)
const dp = new Array(N + 1).fill(0);

// 뒤에서부터 역순으로 반복 (N-1 부터 0까지)
for (let i = N - 1; i >= 0; i--) {
    const time = TP[i][0]; // 상담 걸리는 시간
    const pay = TP[i][1]; // 상담 비용

    // 상담 기간이 퇴사일(N)을 넘기는 경우
    if (i + time > N) {
        dp[i] = dp[i + 1];
    } else {
        // 상담을 안 하는 경우(dp[i+1]) vs 하는 경우(pay + dp[i+time])
        dp[i] = Math.max(dp[i + 1], pay + dp[i + time]);
    }
}

console.log(dp[0]);

```

Python

이친수 구하기



빈출

문제[086] 이진수 구하기

시간 제한 2초 | 난이도 실버 Ⅲ | 백준 온라인 저지 2193번

0과 1로만 이뤄진 수를 '이진수'라고 한다. 이러한 이진수 중 특별한 성질을 갖는 것들이 있는데, 이들을 '이진수^{binary number}'라고 한다. 이진수는 다음의 성질을 만족한다.

- 이진수는 0으로 시작하지 않는다.
- 이진수에서는 1이 두 번 연속으로 나타나지 않는다. 즉, 11을 부분 문자열로 갖지 않는다.

예를 들면 1, 10, 100, 101, 1000, 1001 등이 이진수가 된다. 하지만 0010101이나 101101은 각각 1, 2번 규칙에 위배되므로 이진수가 아니다. $N(1 \leq N \leq 90)$ 이 주어졌을 때 N 자리 이진수의 개수를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 N 이 주어진다.

↑ 출력

1번째 줄에 N 자리 이진수의 개수를 출력한다.

예제 입력 1

6 // N

예제 출력 1

5

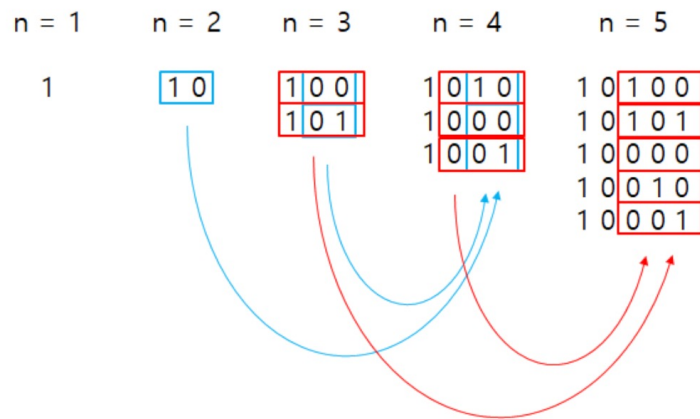
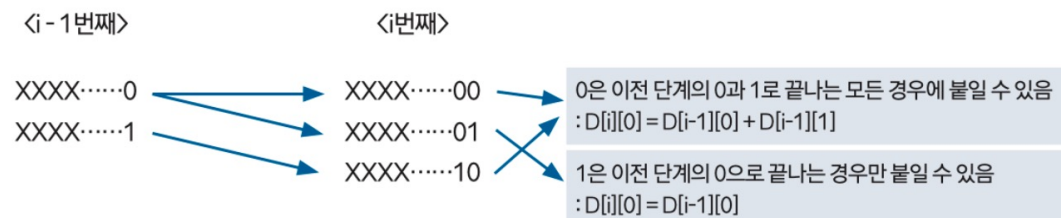


- 1 점화식의 형태와 의미를 도출한다.

$D[i][0]$: i 길이에서 끝이 0으로 끝나는 이진수의 개수

$D[i][1]$: i 길이에서 끝이 1로 끝나는 이진수의 개수

- 2 점화식을 구한다. 1은 두 번 연속 나오지 않는다는 조건이 핵심이다.



Python

```
import sys
input = sys.stdin.readline

n = int(input())

# n 자리 수 만큼 dp 테이블 생성 => n+1
dp = [0] * (n + 1)
dp[1] = 1

for i in range(2, n+1):
    dp[i] = dp[i-2] + dp[i-1]

print(dp[n])
```

Java, JS

이친수 구하기



Java

```
public class P003 { new *
    public static void main(String[] args) { new *
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        // 입력값이 0인 경우에 대한 예외 처리 (문제 조건에 따라 필요할 수 있음)
        if (n == 0) {
            System.out.println(0);
            return;
        }

        // int가 아닌 long 배열 선언
        long[] dp = new long[n + 1];

        dp[1] = 1;
        // dp[0]은 자동으로 0이므로 생략 가능

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 2] + dp[i - 1];
        }

        System.out.println(dp[n]);
        sc.close();
    }
}
```

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim();

const n = Number(input);

if (n === 0) {
    console.log(0);
} else {
    // BigInt 배열 생성 (0n 으로 초기화)
    // 0이 아니라 0n (BigInt 형태의 0)이어야 계산 가능
    const dp = new Array(n + 1).fill(0n);

    dp[1] = 1n; // 1 뒤에 n을 붙여야 BigInt 리터럴

    for (let i = 2; i <= n; i++) {
        dp[i] = dp[i - 2] + dp[i - 1];
    }

    // BigInt는 console.log로 그냥 찍으면 숫자 뒤에 'n'이 붙을 수 있으므로 toString() 사용 권장
    console.log(dp[n].toString());
}
```

Python

2*N 타일 구하기

**문제[087] 2*N 타일 채우기**

시간 제한 1초 | 난이도 실버 III | 백준 온라인 저지 11726번

다음 그림은 2×5 크기의 직사각형을 채운 1가지 방법의 예다. 이렇게 $2 \times n$ 크기의 직사각형을 1×2 또는 2×1 타일로 채우는 방법의 수를 구하는 프로그램을 작성하시오.

**↓ 입력**

1번째 줄에 N 이 주어진다($1 \leq N \leq 1,000$).

↑ 출력

1번째 줄에 $2 \times N$ 크기의 직사각형을 채우는 방법의 수를 10,007로 나눈 나머지를 출력한다.

예제 입력 1

9

예제 출력 1

55

예제 입력 2

2 // N

예제 출력 2

2



Python

2*N 타일 구하기 - 정답

1 점화식의 형태와 의미를 도출한다.

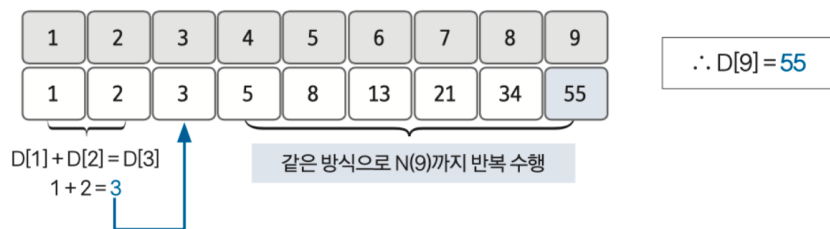
$D[N]$: 길이 N 으로 만들 수 있는 타일의 경우의 수

2 점화식을 구한다.

$D[N] = D[N - 1] + D[N - 2]$ // $D[N - 1]$ 과 $D[N - 2]$ 의 경우의 수의 합이 $D[N]$

3 점화식으로 D 배열을 채운 후 $D[N]$ 의 값을 출력한다.

D 배열을 채울 때마다 문제에서 주어진 값(10,007)으로 % 연산을 수행한다.



Python

```
import sys
input = sys.stdin.readline

n = int(input())
dp = [0] * 1001
dp[1] = 1
dp[2] = 2

for i in range(3, n+1):
    dp[i] = (dp[i-1] + dp[i-2]) % 10007

print(dp[n])
```

Java, JS

2*N 타일 구하기



Java

```
public class P004 { new *
    public static void main(String[] args) { new *
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        // 문제 조건이 N <= 1000 이므로 크기 1001로 고정
        int[] dp = new int[1001];

        // 초기값 설정
        dp[1] = 1;
        dp[2] = 2;

        // 3부터 n까지 반복
        for (int i = 3; i <= n; i++) {
            // 매 계산마다 10007로 나눈 나머지를 저장 (오버플로우 방지)
            dp[i] = (dp[i - 1] + dp[i - 2]) % 10007;
        }

        // 반복문이 끝난 뒤 결과 출력
        System.out.println(dp[n]);

        sc.close();
    }
}
```

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim();

const n = Number(input);

// 크기 1001 배열 생성 (0으로 초기화)
const dp = new Array(1001).fill(0);

// 초기값 설정
dp[1] = 1;
dp[2] = 2;

// 3부터 n까지 반복
for (let i = 3; i <= n; i++) {
    // 매 계산마다 10007로 나눈 나머지 저장
    dp[i] = (dp[i - 1] + dp[i - 2]) % 10007;
}

// 결과 출력
console.log(dp[n]);
```

Python

최장 공통 부분 수열 찾기



핵심

문제[090] 최장 공통 부분 수열 찾기

시간 제한 1초 | 난이도 골드 IV | 백준 온라인 저지 9252번

LCS^{longest common subsequence}: 최장 공통 부분 수열

문제는 두 수열이 주어졌을 때 모두의 부분 수열이 되는 수열 중 가장 긴 것을 찾는 문제다. 예를 들어 ACAYKP와 CAPCAK의 LCS는 ACAK가 된다.

↓ 입력

1번째 줄과 2번째 줄에 두 문자열이 주어진다. 문자열은 알파벳 대문자로만 이뤄져 있으며, 최대 1,000 글자로 이뤄져 있다.

↑ 출력

1번째 줄에 입력으로 주어진 두 문자열의 LCS의 길이, 2번째 줄에 LCS를 출력한다. LCS가 여러 가지 일 때는 아무거나 출력하고, LCS의 길이가 0일 때는 2번째 줄을 출력하지 않는다.

예제 입력 1

ACAYKP
CAPCAK

예제 출력 1

4
ACAК



Python

• 최장 공통 부분 수열 찾기 - 정답

- 1 LCS 점화식을 이용해 값을 채운다. 특정 자리가 가리키는 행과 열의 문자열과 값을 비교해서 값이 같으면 배열의 대각선 왼쪽 위의 값에 1을 더한 값을 저장한다.

$$DP[i][j] = DP[i-1][j-1] + 1$$

비교한 값이 다르면 배열의 왼쪽의 값 중 큰 값을 선택해 저장한다.

$$DP[i][j] = \text{Math.max}(DP[i-1][j], DP[i][j-1])$$

점화식을 이용해 배열을 채우면 다음과 같이 나타낼 수 있다.

두 문자열의 최장 증가 수열의 길이는 4라는 것을 알 수 있다.

	A	C	A	Y	K	P
C	0	1	1	1	1	1
A	1	1	2	2	2	2
P	1	1	2	2	2	3
C	1	2	2	2	2	3
A	1	2	3	3	3	3
K	1	2	3	3	4	4

- 2 LCS 정답을 출력한다. 마지막부터 탐색을 수행한다.

해당 자리에 있는 인덱스 문자열값을 비교해 값이 같으면

최장 증가 수열에 해당하는 문자로 기록하고 왼쪽 대각선으로 이동한다.

비교한 값이 다르면 배열의 왼쪽 위의 값 중 큰 값으로 이동한다.

- ▶ 문자열과 관련된 동적 계획법은 이 문제와 비슷한 방법으로 풀이할 수 있는 경우가 많기 때문에 문제를 꼼꼼하게 숙지하고 실제 코드로 연습해야 합니다.

	A	C	A	Y	K	P
C	0	1	1	1	1	1
A	1	1	2	2	2	2
P	1	1	2	2	2	3
C	1	2	2	2	2	3
A	1	2	3	3	3	3
K	1	2	3	3	4	4

Python

```
str1 = [0] + list(input())
str2 = [0] + list(input())

len_1 = len(str1)
len_2 = len(str2)

array = [['' for _ in range(len_1)] for _ in range(len_2)]

for i in range(1, len_2) :
    for j in range(1, len_1) :
        if str1[j] == str2[i] :
            array[i][j] = array[i-1][j-1] + str1[j]
        else :
            if len(array[i][j-1]) > len(array[i-1][j]) :
                array[i][j] = array[i][j-1]
            else :
                array[i][j] = array[i-1][j]

answer = len(array[-1][-1])
print(answer)
if answer != 0 :
    print(array[-1][-1])
```

Java, JS

최장 공통 부분 수열 찾기



Java

```

public class P005 { new *
    public static void main(String[] args) { new *
        Scanner sc = new Scanner(System.in);

        String str1 = " " + sc.next();
        String str2 = " " + sc.next();

        int len1 = str1.length();
        int len2 = str2.length();

        String[][] dp = new String[len2][len1];

        for (int i = 0; i < len2; i++) {
            for (int j = 0; j < len1; j++) {
                dp[i][j] = "";
            }
        }

        for (int i = 1; i < len2; i++) {
            for (int j = 1; j < len1; j++) {
                // 문자가 같으면: 대각선 위 값 + 현재 문자
                if (str1.charAt(j) == str2.charAt(i)) {
                    dp[i][j] = dp[i - 1][j - 1] + str1.charAt(j);
                }
                // 문자가 다르면: 왼쪽과 위쪽 중 더 긴 문자열 선택
                else {
                    if (dp[i][j - 1].length() > dp[i - 1][j].length()) {
                        dp[i][j] = dp[i][j - 1];
                    } else {
                        dp[i][j] = dp[i - 1][j];
                    }
                }
            }
        }

        String resultStr = dp[len2 - 1][len1 - 1];
        System.out.println(resultStr.length());

        if (resultStr.length() != 0) {
            System.out.println(resultStr);
        }

        sc.close();
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

// 1-based indexing을 위해 앞에 더미 문자('0' 또는 공백) 추가
const str1 = ' ' + input[0].trim();
const str2 = ' ' + input[1].trim();

const len1 = str1.length;
const len2 = str2.length;

// DP 테이블 생성 및 빈 문자열('')로 초기화
// Array.from을 사용하여 2차원 배열 생성
const dp = Array.from({ length: len2 }, () => Array(len1).fill(''));

for (let i = 1; i < len2; i++) {
    for (let j = 1; j < len1; j++) {
        // 문자가 같을 때
        if (str1[j] === str2[i]) {
            dp[i][j] = dp[i - 1][j - 1] + str1[j];
        }
        // 문자가 다를 때
        else {
            if (dp[i][j - 1].length > dp[i - 1][j].length) {
                dp[i][j] = dp[i][j - 1];
            } else {
                dp[i][j] = dp[i - 1][j];
            }
        }
    }
}

// 결과 가져오기 (마지막 요소)
const resultStr = dp[len2 - 1][len1 - 1];
const answer = resultStr.length;

console.log(answer);

if (answer !== 0) {
    console.log(resultStr);
}

```

Python

가장 큰 정사각형 찾기

**문제[091] 가장 큰 정사각형 찾기**

시간 제한 2초 | 난이도 골드 IV | 백준 온라인 저지 1915번

크기가 $n \times m$ 이고 0과 1로만 이루어진 배열이 있다. 이 배열에서 1로 된 가장 큰 정사각형의 크기를 구하는 프로그램을 작성하시오. 예를 들어 다음 그림과 같은 배열에서는 가운데에 있는 2×2 크기의 정사각형이 가장 크다.

0	1	0	0
0	1	1	1
1	1	1	0
0	0	1	0

↓ 입력

1번째 줄에 n, m ($1 \leq n, m \leq 1,000$), 그다음 n 개의 줄에 m 개의 숫자로 배열이 주어진다.

↑ 출력

1번째 줄에 가장 큰 정사각형의 넓이를 출력한다.

예제 입력 1

```
4 4 // n, m
0100
0111
1110
0010
```

예제 출력 1

```
4
```




1 D 배열의 값을 초기화한다.

0	1	0	0
0	1	1	1
1	1	1	0
0	0	1	0

2 점화식을 이용해 D 배열의 값을 새롭게 채운다.

$D[i][j] = \min(D[i-1][j-1], D[i][j-1], D[i-1][j]) + 1$ // 현재 자리의 원래 값이 1
 $D[i][j] = 0$ // 현재 자리의 원래 값이 0

0	1	0	0
0	1	1	1
1	1	1	0
0	0	1	0



0	1	0	0
0	1	1	1
1	1	2	0
0	0	1	0

기존 값이 1이고 위, 왼쪽, 왼쪽 대각선의
최솟값이 1이므로 2로 변경

3 D 배열 중 가장 큰 값의 제곱이 최대 정사각형의 넓이이다.

$$\therefore 2^2 = 4$$

Python

```
import sys

input = sys.stdin.readline

n, m = map(int, input().split())

arr = []
dp = [[0] * m for _ in range(n)]

for _ in range(n):
    arr.append(list(map(int, list(input().rstrip()))))

answer = 0
for i in range(n):
    for j in range(m):
        if i == 0 or j == 0:
            dp[i][j] = arr[i][j]
        elif arr[i][j] == 0:
            dp[i][j] = 0
        else:
            dp[i][j] = min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1]) + 1
        answer = max(dp[i][j], answer)

print(answer * answer)
```

Java, JS

가장 큰 정사각형 찾기



Java

```

public class P006 { new *
    public static void main(String[] args) { new *
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();
        int[][] arr = new int[n][m];
        // DP 테이블
        int[][] dp = new int[n][m];

        for (int i = 0; i < n; i++) {
            // 공백 없이 들어오는 문자열(예: "01001")을 받음
            String line = sc.next();
            for (int j = 0; j < m; j++) {
                // 문자를 숫자로 변환 ('0'을 빼줌)
                arr[i][j] = line.charAt(j) - '0';
            }
        }

        long answer = 0; // 넓이가 커질 수 있으므로 long 사용 권장 (이 문제 범위에선 int도 무관)

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (i == 0 || j == 0) {
                    dp[i][j] = arr[i][j];
                }
                else if (arr[i][j] == 0) {
                    dp[i][j] = 0;
                }
                else {
                    dp[i][j] = Math.min(dp[i - 1][j - 1],
                                         Math.min(dp[i - 1][j], dp[i][j - 1])) + 1;
                }
                answer = Math.max(answer, dp[i][j]);
            }
        }
        System.out.println(answer * answer);
        sc.close();
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

// 첫 줄에서 n, m 추출
const [n, m] = input[0].split(' ').map(Number);

// 2차원 배열 생성
const arr = [];
const dp = Array.from({ length: n }, () => Array(m).fill(0));

// 입력 데이터 파싱 (두 번째 줄부터)
for (let i = 0; i < n; i++) {
    // 문자열을 하나씩 쪼개서 숫자로 변환하여 배열에 담음
    // input[i + 1]을 trim() 하여 \r 등을 제거
    arr.push(input[i + 1].trim().split('').map(Number));
}

let answer = 0;

for (let i = 0; i < n; i++) {
    for (let j = 0; j < m; j++) {
        // 첫 행이나 첫 열인 경우
        if (i === 0 || j === 0) {
            dp[i][j] = arr[i][j];
        }
        // 0인 경우
        else if (arr[i][j] === 0) {
            dp[i][j] = 0;
        }
        // 점화식 적용
        else {
            dp[i][j] = Math.min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1;
        }

        answer = Math.max(answer, dp[i][j]);
    }
}

console.log(answer * answer);

```

Python

행렬 곱 연산의 최소값 찾기



핵심

문제[094] 행렬 곱 연산 횟수의 최소값 구하기

시간 제한 1초 | 난이도 골드 III | 백준 온라인 저지 11049번

크기가 $N \times M$ 인 행렬 A와 $M \times K$ 인 B를 곱할 때 필요한 곱셈 연산 횟수는 총 $N \times M \times K$ 번이다. 행렬 N개를 곱하는 데 필요한 곱셈 연산 횟수는 행렬을 곱하는 순서에 따라 달라진다. 예를 들어 A의 크기가 5×3 이고, B의 크기가 3×2 , C의 크기가 2×6 일 때 행렬의 곱 ABC를 구하는 경우를 생각해 보자. AB를 먼저 곱하고, C를 곱하는 경우 (AB)C에 필요한 곱셈 연산 횟수는 $(5 \times 3 \times 2) + (5 \times 2 \times 6) = 30 + 60 = 90$ 번이다. BC를 먼저 곱한 후 A를 곱하는 경우 A(BC)에 필요한 곱셈 연산 횟수는 $(3 \times 2 \times 6) + (5 \times 3 \times 6) = 36 + 90 = 126$ 번이다. 같은 곱셈이지만, 곱셈을 하는 순서에 따라 곱셈 연산 횟수가 달라진다.

행렬 N개의 크기가 주어졌을 때 모든 행렬을 곱하는 데 필요한 곱셈 연산 횟수의 최소값을 구하는 프로그램을 작성하시오. 단, 입력으로 주어진 행렬의 순서를 바꾸면 안 된다.

↓ 입력

1번째 줄에 행렬의 개수 N ($1 \leq N \leq 500$), 2번째 줄부터 N개 줄에는 행렬의 크기 r 과 c 가 주어진다($1 \leq r, c \leq 500$). 항상 순서대로 곱셈할 수 있는 크기만 입력으로 주어진다.

↑ 출력

1번째 줄에 입력으로 주어진 행렬을 곱하는 데 필요한 곱셈 연산 횟수의 최소값을 출력한다. 정답은 231 - 1보다 작거나 같은 자연수다. 또한 최악의 순서로 연산해도 연산 횟수가 231 - 1보다 작거나 같다.

예제 입력 1

```
3 // 행렬 개수
5 3
3 2
2 6
```

예제 출력 1

```
90
```



Python

행렬 곱 연산의 최소값 찾기 - 정답

1 행렬 구간에 행렬이 1개일 때 0을 리턴한다.

행렬 구간에 행렬이 2개일 때 '앞 행렬의 row 값 * 뒤 행렬의 row 값 * 뒤 행렬의 column 값' 을 리턴한다.

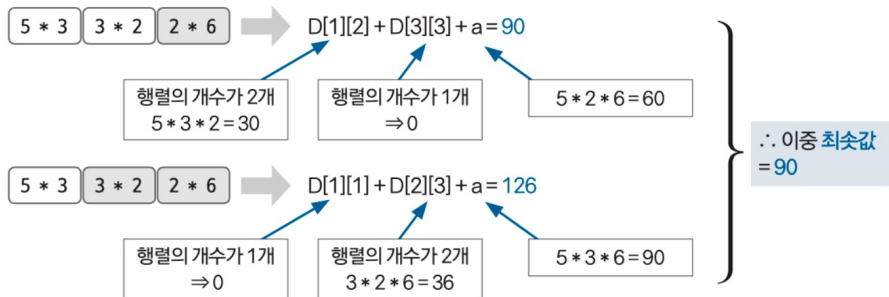
행렬 구간에 행렬이 3개 이상일 때는 다음 조건식의 결과값을 리턴한다.

행렬 구간에 행렬이 3개 이상일 때 조건식

// s: 시작 인덱스 e: 종료 인덱스

for(i를 시작 행렬부터 종료 행렬까지 반복하기)

min = Math.min(min, D[s][i] + D[i+1][e] + a(s 행렬의 row * i + 1 행렬의 row * e 행렬의 column))



Python

```
N = int(input())
matrix = []
for _ in range(N):
    matrix.append(list(map(int, input().split())))
dp = [[0 for _ in range(N)] for _ in range(N)]

for i in range(1, N): # 몇 번째 대각선?
    for j in range(0, N-i): # 대각선에서 몇 번째 열?

        if i == 1: # 차이가 1밖에 나지 않는 칸
            dp[j][j+i] = matrix[j][0] * matrix[j][1] * matrix[j+i][1]
            continue

        dp[j][j+i] = 2**32 # 최댓값을 미리 넣어줌
        for k in range(j, j+i):
            dp[j][j+i] = min(dp[j][j+i],
                              dp[j][k] + dp[k+1][j+i] + matrix[j][0] * matrix[k][1] * matrix[j+i][1])

print(dp[0][N-1]) # 맨 오른쪽 위
```

Java, JS

행렬 곱 연산의 최소값 찾기



Java

```

public class P007 {
    new *
    public static void main(String[] args) {
        new *
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        int[][] arr = new int[N][2];
        for (int i = 0; i < N; i++) {
            arr[i][0] = sc.nextInt();
            arr[i][1] = sc.nextInt();
        }
        int[][] dp = new int[N][N];

        for (int term = 1; term < N; term++) {
            // start: 시작 행렬 인덱스 - 파이썬의 range(N)
            for (int start = 0; start < N; start++) {

                // 범위를 벗어나면 해당 term 루프는 더 이상 볼 필요 없음 (break)
                if (start + term >= N) {
                    break;
                }

                int end = start + term; // 끝 행렬 인덱스

                // 최소값을 구하기 위해 큰 값으로 초기화
                dp[start][end] = Integer.MAX_VALUE;

                // t: 분할 지점 - 파이썬의 range(start, start + term)
                // start ~ t (왼쪽) / t+1 ~ end (오른쪽) 으로 나눔
                for (int t = start; t < end; t++) {
                    int cost = dp[start][t] + dp[t + 1][end]
                        + arr[start][0] * arr[t][1] * arr[end][1];

                    dp[start][end] = Math.min(dp[start][end], cost);
                }
            }
        }
        System.out.println(dp[0][N - 1]);
        sc.close();
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const N = Number(input[0]);
const arr = [];

// 행렬 정보 파싱
for (let i = 1; i <= N; i++) {
    arr.push(input[i].trim().split(' ').map(Number));
}

// DP 테이블 생성 (0으로 초기화)
const dp = Array.from({ length: N }, () => Array(N).fill(0));

// term: 구간의 간격
for (let term = 1; term < N; term++) {
    // start: 시작점
    for (let start = 0; start < N; start++) {

        // 범위 체크
        if (start + term >= N) {
            break;
        }

        const end = start + term; // 끝점

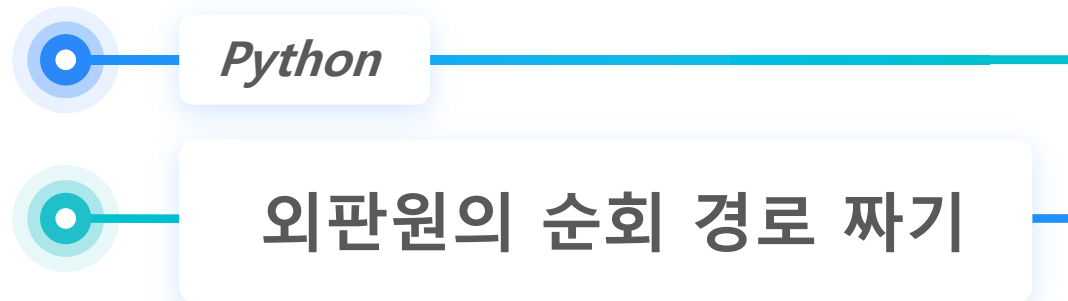
        // 최소값 갱신을 위해 무한대로 초기화
        dp[start][end] = Infinity; // 파이썬의 int(1e9) 대신 Infinity 사용 권장

        // t: 자르는 지점
        for (let t = start; t < end; t++) {
            // 점화식: 왼쪽 비용 + 오른쪽 비용 + 합치는 비용
            // arr[start][0]: 전체 구간의 행 개수
            // arr[t][1]: 앞부분의 열 개수 (뒷부분의 행 개수와 같음)
            // arr[end][1]: 전체 구간의 열 개수
            const cost = dp[start][t] + dp[t + 1][end]
                + arr[start][0] * arr[t][1] * arr[end][1];

            dp[start][end] = Math.min(dp[start][end], cost);
        }
    }
}

console.log(dp[0][N - 1]);

```



핵심

문제[095] 외판원의 순회 경로 짜기

시간 제한 1초 | 난이도 골드 I | 백준 온라인 저지 2098번

1번부터 N번까지 번호가 매겨져 있는 도시들이 있고, 도시들 사이에는 길이 있다(길이 없을 수도 있다). 이제 한 외판원이 어느 한 도시에서 출발해 N개의 도시를 모두 거쳐 다시 원래의 도시로 돌아오는 순회 여행 경로를 계획하려고 한다. 단, 한 번 갔던 도시로는 다시 갈 수 없다(맨 마지막에 여행을 출발했던 도시로 돌아오는 것은 예외). 이런 여행 경로에는 여러 가지가 있을 수 있는데, 가장 적은 비용을 들이는 여행 계획을 세우고자 한다. 각 도시 간에 이동하는 데 드는 비용은 행렬 $W[i][j]$ 형태로 주어진다. $W[i][j]$ 는 도시 i에서 도시 j로 가기 위한 비용을 나타낸다. 비용은 대칭적이지 않다. 즉, $W[i][j]$ 는 $W[j][i]$ 와 다를 수 있다. 모든 도시 간의 비용은 양의 정수다. $W[i][i]$ 는 항상 0이다. 경우에 따라 도시 i에서 도시 j로 갈 수 없을 때도 있고, 이럴 경우 $W[i][j] = 0$ 이라고 가정한다. N과 비용 행렬이 주어졌을 때 가장 적은 비용을 들이는 외판원의 순회 여행 경로를 구하는 프로그램을 작성하시오.

예제 입력 1

```
4 // N
0 10 15 20
5 0 9 10
6 13 0 12
8 8 9 0
```

예제 출력 1

35

↓ 입력

1번째 줄에 도시의 수 N이 주어진다($2 \leq N \leq 16$). 다음 N개의 줄에는 비용 행렬이 주어진다. 각 행렬의 성분은 1,000,000 이하의 양의 정수이며, 갈 수 없을 때는 0이 주어진다. $W[i][j]$ 는 도시 i에서 j로 가기 위한 비용을 나타낸다. 항상 순회할 수 있을 때만 입력으로 주어진다.

↑ 출력

1번째 줄에 외판원의 순회에 필요한 최소 비용을 출력한다.



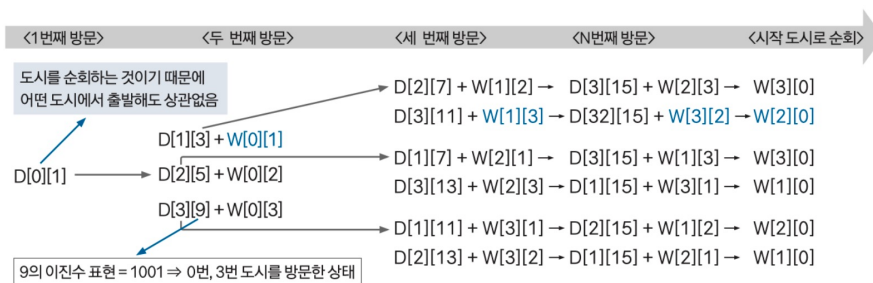
- 1 c번 도시에서 v리스트 도시를 방문한 후 남은 모든 도시를 순회하기 위한 최소 비용은 현재 방문하지 않은 모든 도시에 대해 반복하고, 방문하지 않은 도시를 i라고 했을 때 다음과 같다. $W[c][i]$ 는 도시 c에서 도시 i로 가기 위한 비용을 나타낸다.

$$D[c][v] = \text{Math.min}(D[c][v], D[i][v] + (1 \ll i) + W[c][i])$$

- 2 W 배열을 저장한다.

	0	1	2	3
0	0	10	15	20
1	5	0	9	10
2	6	13	0	12
3	8	8	9	0

- 3 점화식으로 정답을 구한다.



- 4 최솟값을 정답으로 출력한다.

$$\therefore W[0][1] + W[1][3] + W[3][2] + W[2][0] = 10 + 10 + 9 + 6 = 35$$

Python

```
import sys
N = int(input())
world = []
for _ in range(N):
    world.append(list(map(int, sys.stdin.readline().split())))

dp = {}

def DFS(now, visited):
    # 모든 도시를 방문한 경우
    if visited == (1 << N) - 1:
        # 다시 출발 도시로 갈 수 있는 경우 출발 도시까지의 비용 반환
        if world[now][0]:
            return world[now][0]
        else:
            # 갈 수 없는 경우 무한대 반환 (이 경로가 최소비용으로 채택되지 않게)
            return int(1e9)

    # 이전에 계산된 경우 결과 반환
    if (now, visited) in dp:
        return dp[(now, visited)] # now까지 방문한 최소 비용

    min_cost = int(1e9)
    for next in range(1, N):
        # 비용이 0이어서 갈 수 없거나, 이미 방문한 루트면 무시
        if world[now][next] == 0 or visited & (1 << next):
            continue
        cost = DFS(next, visited | (1 << next)) + world[now][next]
        min_cost = min(cost, min_cost)

    dp[(now, visited)] = min_cost # 현재 경우에서 최소 비용이 드는 루트의 비용 저장
    return min_cost # 비용 리턴

print(DFS(0, 1)) # now: 0번째 도시부터 방문, visited: 0번째 도시 방문 처리
```

Java, JS

외판원의 순회 경로 짜기



Java

```
public class P088 { new *
    static int N; 9 usages
    static int[][] world; 6 usages
    static int[][] dp; 8 usages
    static final int INF = 1000000000; // 10억 (무한대 대용) 3 usages

    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        N = Integer.parseInt(br.readLine());
        world = new int[N][N];
        dp = new int[N][1 << N];

        for (int i = 0; i < N; i++) {
            st = new StringTokenizer(br.readLine());
            for (int j = 0; j < N; j++) {
                world[i][j] = Integer.parseInt(st.nextToken());
            }
            Arrays.fill(dp[i], -1);
        }
        System.out.println(dfs(0, 1));
    }

    static int dfs(int now, int visited) { 2 usages new *
        // 모든 도시를 방문했을 경우 ((1<<N) - 1 은 모든 비트가 1인 상태)
        if (visited == (1 << N) - 1) {
            // 현재 도시에서 출발 도시(0)로 돌아갈 수 있으면 그 비용 리턴
            if (world[now][0] != 0) {
                return world[now][0];
            }
            // 돌아갈 수 없으면 무한대 리턴
            return INF;
        }
        if (dp[now][visited] != -1) {
            return dp[now][visited];
        }
        dp[now][visited] = INF; // 최소값을 구하기 위해 초기화
        for (int next = 0; next < N; next++) {
            if (world[now][next] == 0 || (visited & (1 << next)) != 0) {
                continue;
            }
            int cost = dfs(next, visited | (1 << next));
            if (cost != INF) {
                dp[now][visited] = Math.min(dp[now][visited], cost + world[now][next]);
            }
        }
        return dp[now][visited];
    }
}
```

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const N = Number(input[0]);
const world = [];

for (let i = 1; i <= N; i++) {
    world.push(input[i].trim().split(' ').map(Number));
}

const dp = Array.from({ length: N }, () => Array(1 << N).fill(-1));
const INF = 1e9; // 10억

function dfs(now, visited) {
    // 모든 도시 방문 완료 (비트가 모두 1)
    if (visited === (1 << N) - 1) {
        // 출발점(0)으로 돌아갈 수 있는 경우
        if (world[now][0] !== 0) {
            return world[now][0];
        }
        return INF; // 불가
    }

    if (dp[now][visited] !== -1) {
        return dp[now][visited];
    }

    let min_cost = INF;
    for (let next = 1; next < N; next++) {
        // 갈 수 없거나(0), 이미 방문했다면 스킵
        // (visited & (1 << next)) !== 0 은 방문했다는 뜻
        if (world[now][next] === 0 || (visited & (1 << next)) !== 0) {
            continue;
        }

        const cost = dfs(next, visited | (1 << next));

        // 유효한 경로라면 비용 갱신
        if (cost !== INF) {
            min_cost = Math.min(min_cost, cost + world[now][next]);
        }
    }

    dp[now][visited] = min_cost;
    return min_cost;
}

console.log(dfs(0, 1));
```

Python

선분 방향 구하기

**문제[097] 선분 방향 구하기**

시간 제한 1초 | 난이도 골드 V | 백준 온라인 저지 11758번

2차원 좌표 평면 위에 있는 점 3개 P_1, P_2, P_3 이 주어진다. P_1, P_2, P_3 을 순서대로 이은 선분이 어떤 방향을 이루고 있는지 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 P_1 의 (x_1, y_1) , 2번째 줄에 P_2 의 (x_2, y_2) , 3번째 줄에 P_3 의 (x_3, y_3) 가 주어진다($-10,000 \leq x_1, y_1, x_2, y_2, x_3, y_3 \leq 10,000$). 모든 좌표는 정수다. P_1, P_2, P_3 의 좌표는 서로 다르다.

↑ 출력

P_1, P_2, P_3 을 순서대로 이은 선분이 반시계 방향이면 1, 시계 방향이면 -1, 일직선이면 0을 출력한다.

예제 입력 1

```
1 1
5 5
7 3
```

예제 출력 1

```
-1
```

예제 입력 2

```
1 1
3 3
5 5
```

예제 출력 2

```
0
```

예제 입력 3

```
1 1
7 3
5 5
```

예제 출력 3

```
1
```



- 1 P_1, P_2, P_3 3개의 점을 입력받아 변수에 저장하고, ccw를 계산한다.

$$\begin{aligned} \text{CCW} &= (X_1Y_2 + X_2Y_3 + X_3Y_1) - (X_2Y_1 + X_3Y_2 + X_1Y_3) \\ &= (1 * 5 + 5 * 3 + 7 * 1) - (5 * 1 + 7 * 5 + 1 * 3) \\ &= 27 - 43 = -16 \end{aligned}$$

- 2 CCW 결괏값에 따라 정답을 출력한다.

$\therefore -16$ 은 0보다 작으므로 -1 출력

Python

```
import sys
input = sys.stdin.readline
def ccw(x1, y1, x2, y2, x3, y3):
    return x1*y2 + x2*y3 + x3*y1 - x2*y1 - x3*y2 - x1*y3
x1, y1 = map(int, input().split())
x2, y2 = map(int, input().split())
x3, y3 = map(int, input().split())
res=ccw(x1, y1, x2, y2, x3, y3)
if res>0:
    print(1)
elif res<0:
    print(-1)
else:
    print(0)
```


Java, JS

선분 방향 구하기



Java

```
public class P010 { new *
    public static void main(String[] args) { new *
        Scanner sc = new Scanner(System.in);

        // P1 입력
        int x1 = sc.nextInt();
        int y1 = sc.nextInt();

        // P2 입력
        int x2 = sc.nextInt();
        int y2 = sc.nextInt();

        // P3 입력
        int x3 = sc.nextInt();
        int y3 = sc.nextInt();

        // CCW 공식 계산 (외적값)
        // (x1y2 + x2y3 + x3y1) - (x2y1 + x3y2 + x1y3)
        int res = (x1 * y2 + x2 * y3 + x3 * y1) - (x2 * y1 + x3 * y2 + x1 * y3);

        // 결과 판별
        if (res > 0) {
            System.out.println(1); // 반시계
        } else if (res < 0) {
            System.out.println(-1); // 시계
        } else {
            System.out.println(0); // 일직선
        }

        sc.close();
    }
}
```

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

// 각 줄을 공백 기준으로 잘라 숫자 배열로 변환
const [x1, y1] = input[0].trim().split(' ').map(Number);
const [x2, y2] = input[1].trim().split(' ').map(Number);
const [x3, y3] = input[2].trim().split(' ').map(Number);

// CCW 공식 계산
const res = (x1 * y2 + x2 * y3 + x3 * y1) - (x2 * y1 + x3 * y2 + x1 * y3);

// 결과 판별
if (res > 0) {
    console.log(1);
} else if (res < 0) {
    console.log(-1);
} else {
    console.log(0);
}
```

Python

선분의 교차 여부 구하기

**문제[098] 선분의 교차 여부 구하기**

시간 제한 0.25초 | 난이도 골드 II | 백준 온라인 저지 17387번

2차원 좌표 평면 위의 두 선분 L_1 , L_2 가 주어졌을 때 두 선분이 교차하는지 아닌지 구해 보자. 한 선분의 끝점이 다른 선분이나 끝점 위에 있으면 교차하는 것이다. L_1 의 양끝점은 (x_1, y_1) , (x_2, y_2) , L_2 의 양끝점은 (x_3, y_3) , (x_4, y_4) 이다.

↓ 입력

1번째 줄에 L_1 의 양끝점 x_1, y_1, x_2, y_2 , 2번째 줄에 L_2 의 양끝점 x_3, y_3, x_4, y_4 가 주어진다.

↑ 출력

L_1 과 L_2 가 교차하면 1, 아니면 0을 출력한다($-1,000,000 \leq x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4 \leq 1,000,000$)

($x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4$ 는 정수).

예제 입력 1

1 1 5 5
1 5 5 1

예제 출력 1

1

예제 입력 2

1 1 5 5
6 10 10 6

예제 출력 2

0

예제 입력 3

1 1 5 5
5 5 1 1

예제 출력 3

1

예제 입력 1

1 1 5 5
3 3 5 5

예제 출력 1

1

예제 입력 2

1 1 5 5
3 3 1 3

예제 출력 2

1

예제 입력 3

1 1 5 5
5 5 9 9

예제 출력 3

1

예제 입력 1

1 1 5 5
6 6 9 9

예제 출력 1

0

예제 입력 2

1 1 5 5
5 5 1 5

예제 출력 2

1

예제 입력 3

1 1 5 5
6 6 1 5

예제 출력 3

0



1 두 선분과 관련된 CCW값의 곱을 구한다.

$$x_1 = 1, y_1 = 1, x_2 = 5, y_2 = 5$$

$$x_3 = 1, y_3 = 5, x_4 = 5, y_4 = 1$$

각 점을 A, B, C, D로 정했을 때 CCW값을 구하면

$$CCW(ABC) = (1 * 5 + 5 * 5 + 1 * 1) - (5 * 1 + 1 * 5 + 1 * 5) = 16$$

$$CCW(ABD) = (1 * 5 + 5 * 1 + 5 * 1) - (5 * 1 + 5 * 5 + 1 * 1) = -16$$

$$CCW(CDA) = (1 * 1 + 5 * 1 + 1 * 5) - (5 * 5 + 1 * 1 + 1 * 1) = -16$$

$$CCW(CDB) = (1 * 1 + 5 * 5 + 5 * 5) - (5 * 5 + 5 * 1 + 1 * 5) = 16$$

2 결과에 따른 선분 교차 여부를 확인한다.

$$CCW(ABC) * CCW(ABD) = (16) * (-16) = \text{음수}$$

$$CCW(CDA) * CCW(CDB) = (-16) * (16) = \text{음수}$$

⇒ 곱의 결과값이 모두 음수 ⇒ 선분이 교차함 ⇒ ∴ 1 출력

Python

```
import sys
input = sys.stdin.readline

def solution():
    ccw123 = ccw(x1, y1, x2, y2, x3, y3)
    ccw124 = ccw(x1, y1, x2, y2, x4, y4)
    ccw341 = ccw(x3, y3, x4, y4, x1, y1)
    ccw342 = ccw(x3, y3, x4, y4, x2, y2)

    # 평행
    if ccw123*ccw124 == 0 and ccw341*ccw342 == 0:
        if mx1 <= mx4 and mx3 <= mx2 and my1 <= my4 and my3 <= my2:
            return 1

    # 교차
    else:
        if ccw123*ccw124 <= 0 and ccw341*ccw342 <= 0:
            return 1

    return 0

def ccw(x1, y1, x2, y2, x3, y3):
    return (x2-x1)*(y3-y1) - (y2-y1)*(x3-x1)

if __name__ == '__main__':
    x1, y1, x2, y2 = list(map(int, input().split()))
    x3, y3, x4, y4 = list(map(int, input().split()))

    mx1, my1, mx2, my2 = min(x1, x2), min(y1, y2), max(x1, x2), max(y1, y2)
    mx3, my3, mx4, my4 = min(x3, x4), min(y3, y4), max(x3, x4), max(y3, y4)

    print(solution())
```

Java, JS

선분의 교차 여부 구하기



Java

```
public class P011 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        long x1 = Long.parseLong(st.nextToken());
        long y1 = Long.parseLong(st.nextToken());
        long x2 = Long.parseLong(st.nextToken());
        long y2 = Long.parseLong(st.nextToken());

        st = new StringTokenizer(br.readLine());
        // 선분 2 (x3, y3, x4, y4)
        long x3 = Long.parseLong(st.nextToken());
        long y3 = Long.parseLong(st.nextToken());
        long x4 = Long.parseLong(st.nextToken());
        long y4 = Long.parseLong(st.nextToken());

        System.out.println(solution(x1, y1, x2, y2, x3, y3, x4, y4));
    }

    static int ccw(long x1, long y1, long x2, long y2, long x3, long y3) { 4 usages new *
        long result = (x2 - x1) * (y3 - y1) - (y2 - y1) * (x3 - x1);
        if (result > 0) return 1; // 반시계
        if (result < 0) return -1; // 시계
        return 0; // 일직선
    }

    static int solution(long x1, long y1, long x2, long y2, long x3, long y3, long x4, long y4) { 1 usage new *
        int ccw123 = ccw(x1, y1, x2, y2, x3, y3);
        int ccw124 = ccw(x1, y1, x2, y2, x4, y4);
        int ccw341 = ccw(x3, y3, x4, y4, x1, y1);
        int ccw342 = ccw(x3, y3, x4, y4, x2, y2);
        boolean isStraight = (ccw123 * ccw124 == 0) && (ccw341 * ccw342 == 0);

        if (isStraight) {
            // 포개짐 여부 확인 (Bounding Box Check)
            // 각 선분의 양 끝점을 정렬하여 min, max를 구함
            if (Math.min(x1, x2) <= Math.max(x3, x4) &&
                Math.min(x3, x4) <= Math.max(x1, x2) &&
                Math.min(y1, y2) <= Math.max(y3, y4) &&
                Math.min(y3, y4) <= Math.max(y1, y2)) {
                return 1;
            }
        } else {
            // 등호(<=)가 들어가는 이유는 끝점에서 만나는 경우도 교차로 치기 때문
            if (ccw123 * ccw124 <= 0 && ccw341 * ccw342 <= 0) {
                return 1;
            }
        }
        return 0;
    }
}
```

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');
const [x1, y1, x2, y2] = input[0].trim().split(/\s+/).map(Number);
const [x3, y3, x4, y4] = input[1].trim().split(/\s+/).map(Number);

function ccw(x1, y1, x2, y2, x3, y3) {
    // 좌표값이 매우 클 경우 BigInt를 써야 하지만, 문제 조건(100만 이하)에서는
    // 일반 연산 후 부호만 판별해도 무방합니다.
    const result = (x2 - x1) * (y3 - y1) - (y2 - y1) * (x3 - x1);

    if (result > 0) return 1;
    if (result < 0) return -1;
    return 0;
}

function solution() {
    const ccw123 = ccw(x1, y1, x2, y2, x3, y3);
    const ccw124 = ccw(x1, y1, x2, y2, x4, y4);
    const ccw341 = ccw(x3, y3, x4, y4, x1, y1);
    const ccw342 = ccw(x3, y3, x4, y4, x2, y2);

    if (ccw123 * ccw124 === 0 && ccw341 * ccw342 === 0) {
        // Bounding Box 비교 (검치는지 확인)
        const mx1 = Math.min(x1, x2);
        const my1 = Math.min(y1, y2);
        const mx2 = Math.max(x1, x2);
        const my2 = Math.max(y1, y2);

        const mx3 = Math.min(x3, x4);
        const my3 = Math.min(y3, y4);
        const mx4 = Math.max(x3, x4);
        const my4 = Math.max(y3, y4);

        if (mx1 <= mx4 && mx3 <= mx2 && my1 <= my4 && my3 <= my2) {
            return 1;
        }
    } else {
        if (ccw123 * ccw124 <= 0 && ccw341 * ccw342 <= 0) {
            return 1;
        }
    }
    return 0;
}

console.log(solution());
```

감사합니다!